

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
from utn.frsn import TecnicaturaProgramacion
```

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL SAN NICOLÁS
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

MATERIA: **Programación 1 (Python)**
TRABAJO PRÁCTICO: **Trabajo Práctico Integrador (TPI)**

```
class ProyectoIntegrador(Materia):
```

```
    title = "Sistema de Gestión y Análisis Demográfico de  
            Países"
```

```
    def integrantes():
```

```
        return ["Lautaro Nuñez (Comisión 15)"  
                "Braian Cortez (Comisión 18)"]
```

```
        # FECHA DE ENTREGA
```

```
print('15 / 06 / 2026')
```

ÍNDICE

1. Introducción y Objetivos	Página 3
2. Marco Teórico Conceptual	Página 4
2.1 Estructuras Secuenciales y Condicionales	Página 4
2.2 Estructuras Repetitivas e Iteración	Página 4
2.3 Modularización mediante Funciones	Página 4
2.4 Estructuras de Datos Complejas (Listas y Diccionarios)	Página 5
2.5 Manejo de Archivos Planos (CSV) y Errores	Página 5
3. Decisiones Técnicas y Diseño del Sistema	Página 5
3.1 Arquitectura del Código y Diagrama de Flujo	Página 5
3.2 Guía de Uso del Sistema y Capturas de Pantalla	Página 7
4. Dificultades Encontradas y Resolución	Página 9
5. Conclusiones y Aprendizajes	Página 11
6. Bibliografía y Webgrafía	Página 11

1. Introducción y Objetivos

Introducción

En el ámbito de la ingeniería de software y el análisis de datos, la gestión eficiente de la información resulta fundamental para la toma de decisiones. El presente trabajo práctico aborda el desarrollo de un sistema informático de consola diseñado para administrar, filtrar y analizar datos demográficos y geográficos de países. La problemática principal que resuelve este software es la centralización y manipulación ordenada de registros que incluyen nombres, poblaciones, superficies territoriales y pertenencia continental. A través de una interfaz interactiva de consola, el sistema permite superar las limitaciones del procesamiento manual de datos, garantizando la consistencia de la información y agilizando la obtención de métricas estadísticas globales.

Objetivos del Proyecto

- **Aplicación de Competencias de Programación 1:** Implementar de forma integrada los conceptos de lógica algorítmica, estructuras de control y tipos de datos complejos provistos por la cátedra.
- **Modularización y Buenas Prácticas:** Desarrollar un código limpio y legible basado en el principio de descomposición funcional, donde cada bloque de código posea una responsabilidad única.
- **Persistencia de Datos:** Asegurar que la información modificada por el usuario trascienda la ejecución del programa mediante el uso de almacenamiento secundario.
- **Trabajo Colaborativo Profesional:** Utilizar herramientas de control de versiones distribuidas para coordinar el desarrollo del software de manera equitativa entre los integrantes.

2. Marco Teórico Conceptual

2.1 Estructuras Secuenciales y Condicionales

Las estructuras secuenciales representan el flujo lineal de un programa, donde una instrucción se ejecuta inmediatamente después de la anterior. Las estructuras condicionales (instrucciones `if`, `elif` y `else`) permiten desviar el flujo de ejecución basándose en la evaluación de una condición lógica. En este sistema, las condicionales resultan críticas para la validación de datos de entrada, impidiendo el procesamiento de valores vacíos o numéricos negativos, y guiando las decisiones del menú principal.

2.2 Estructuras Repetitivas e Iteración

Los bucles lógicos permiten ejecutar un bloque de código múltiples veces. El sistema emplea el bucle indeterminado `while` para mantener el menú interactivo en constante ejecución hasta que el usuario decida finalizarlo explícitamente. Asimismo, se utilizan estructuras determinadas `for` para iterar y recorrer las colecciones de datos almacenadas en memoria, permitiendo evaluar las propiedades de cada registro de forma individual durante las operaciones de búsqueda y filtrado.

2.3 Modularización mediante Funciones

La modularización es la técnica de diseño de software que consiste en dividir un problema complejo en subproblemas más pequeños e independientes. En Python, esto se logra mediante la declaración de funciones (`def`). Cada función actúa como una caja negra que recibe parámetros, procesa información y devuelve un resultado. Esta práctica facilita la reutilización de código, simplifica la detección de errores de sintaxis o lógica, y mejora el mantenimiento general del sistema.

Para profundizar los criterios de modularidad, el sistema se estructuró bajo el enfoque de diseño arquitectónico por capas. Esto consistió en organizar de forma secuencial las funciones del sistema dentro de un único archivo principal (`integrador.py`), estructurando bloques independientes para la captura de datos, la lógica de validación algorítmica y la visualización de menús, incrementando la visibilidad global y la mantenibilidad del código.

2.4 Estructuras de Datos Complejas (Listas y Diccionarios)

Para representar los países en la memoria RAM, se optó por una estructura combinada de listas y diccionarios:

- **Diccionarios:** Son colecciones de datos indexadas por claves, asociadas a un valor unívoco. Permiten agrupar las distintas propiedades de un objeto en una sola variable. En este TPI, cada país se define como un diccionario con las llaves nombre, poblacion, superficie y continente.
- **Listas:** Son colecciones ordenadas y mutables de elementos. Se utilizan para almacenar la totalidad de los diccionarios de los países, estructurando una base de datos dinámica en memoria que puede expandirse o modificarse en tiempo de ejecución.

2.5 Manejo de Archivos Planos (CSV) y Errores

La memoria RAM es volátil; los datos se pierden al cerrarse el programa. Para lograr persistencia, se implementó el uso de archivos CSV (Valores Separados por Comas), un formato estándar de texto plano que organiza la información en filas y columnas. El programa interactúa con el sistema operativo para leer el archivo al iniciar y sobrescribirlo al finalizar.

A su vez, este proceso se protege mediante bloques de control de excepciones (try y except). El manejo de errores evita que el programa sufra una finalización abrupta (un fallo crítico) en caso de que el archivo no exista, se encuentre bloqueado o contenga tipos de datos corruptos.

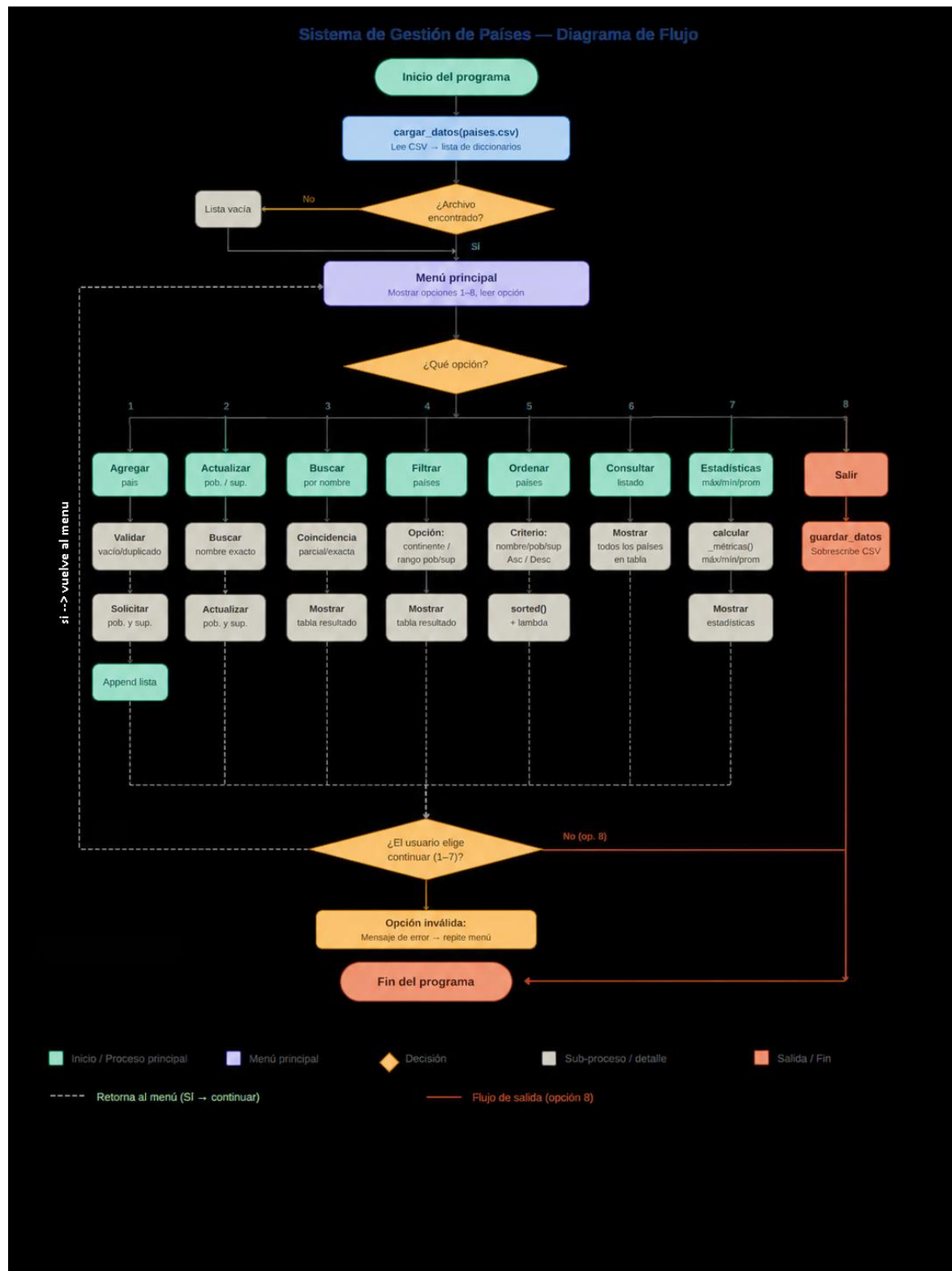
3. Decisiones Técnicas y Diseño del Sistema

3.1 Arquitectura del Código y Diagrama de Flujo

El diseño del sistema adopta una arquitectura de archivo único centrada en el script integrador.py. La solución consolida en un mismo entorno tanto las rutinas de persistencia e interacción con el archivo CSV, como los subprocesos de lógica demográfica y el menú principal. Esta organización estratégica asegura una ejecución

secuencial fluida, facilitando que el intérprete de Python administre de forma centralizada las variables globales y el flujo lógico sin dependencias de módulos externos.

Diagrama de Flujo del Sistema



3.2 Guía de Uso del Sistema y Capturas de Pantalla

A continuación, se presentan los flujos principales de uso del sistema con ejemplos de ejecución real en consola.

Inicio del Sistema

Para ejecutar el programa, el usuario debe abrir una terminal en la carpeta del proyecto y ejecutar el siguiente comando:

```
python integrador.py
```

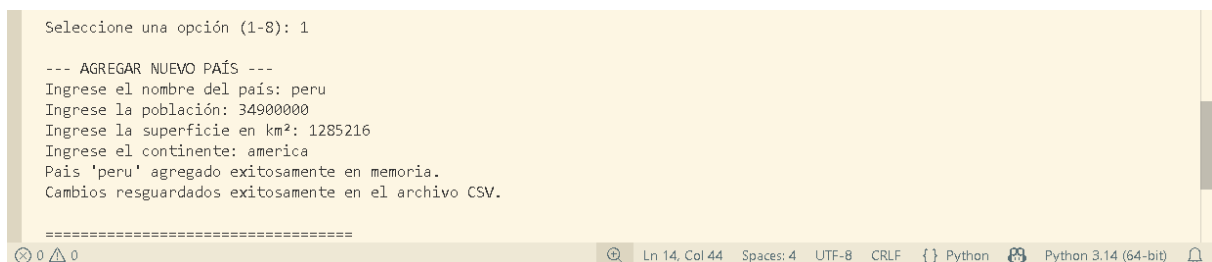
Al iniciarse, el sistema carga automáticamente los datos del archivo `países.csv` y presenta el menú principal de opciones.



```
PS C:\Users\lautaro\AppData\Local\Programs\Microsoft VS Code> C:\Users\lautaro\AppData\Local\Programs\Python\Python314\python.exe  
"C:/Users/lautaro/Desktop/programacion/programacion 1/integrador/integrador.py"  
Alerta: No se encontro el archivo 'países.csv'.  
Se iniciara con una lista vacia.  
  
=====
          SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Agregar un país
2. Actualizar población y superficie
3. Buscar un país por nombre
4. Filtrar países
5. Ordenar países
6. Consultar Listado de Países
7. Mostrar estadísticas
8. Salir
=====
Seleccione una opción (1-8):
```

Agregar un País

Seleccionando la opción 1 del menú, el sistema solicita al usuario los datos del nuevo país: nombre, población, superficie y continente. El sistema valida que ningún campo esté vacío y que los valores numéricos sean enteros positivos. Además, previene la inserción de países duplicados mediante comparación normalizada (sin distinción de mayúsculas).



```
Seleccione una opción (1-8): 1  
  
--- AGREGAR NUEVO PAÍS ---  
Ingrese el nombre del país: peru  
Ingrese la población: 34900000  
Ingrese la superficie en km²: 1285216  
Ingrese el continente: america  
País 'peru' agregado exitosamente en memoria.  
Cambios resguardados exitosamente en el archivo CSV.  
  
=====
```

Buscar, Filtrar y Ordenar

La opción 3 permite buscar por nombre con coincidencia parcial. Las opciones 4 y 5 ofrecen filtrado por continente, rango de población o superficie, y ordenamiento por cualquier campo en sentido ascendente o descendente. Los resultados se presentan en formato de tabla alineada en consola.

```
Seleccione una opción (1-8): 3

--- BUSCAR PAÍS ---
Ingrese el término a buscar: argentina

Resultados encontrados (1):
Nombre      | Población  | Superficie | Continente
-----
Argentina   | 45376763   | 2780400    | América

=====

Seleccione una opción (1-8): 4

--- FILTRAR PAÍSES ---
1. Filtrar por Continente
2. Filtrar por Rango de Población
3. Filtrar por Rango de Superficie
Seleccione una opción de filtrado (1-3): 1
Ingrese el continente a filtrar: america

Resultados del filtro (2):
Nombre      | Población  | Superficie | Continente
-----
Argentina   | 45376763   | 2780400    | América
Brasil      | 213993437  | 8515767    | América

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====

Seleccione una opción (1-8): 5

--- ORDENAR PAÍSES ---
1. Ordenar por Nombre
2. Ordenar por Población
3. Ordenar por Superficie
Seleccione una opción de ordenamiento (1-3): 2

1. Ascendente (Menor a Mayor / A-Z)
2. Descendente (Mayor a Menor / Z-A)
Seleccione el sentido del orden (1-2): 1

Lista ordenada:
Nombre      | Población  | Superficie | Continente
-----
Argentina   | 45376763   | 2780400    | América
Alemania    | 83149300   | 357022     | Europa
Japón       | 125800000  | 377975     | Asia
Brasil      | 213993437  | 8515767    | América

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
```

Estadísticas

La opción 7 calcula y muestra: el país con mayor y menor población, el promedio de población y superficie globales, y la cantidad de países registrados por continente.


```
Seleccione una opción (1-8): 7

--- ESTADÍSTICAS DEL SISTEMA ---
País con mayor población: Brasil (213993437 hab.)
País con menor población: Argentina (45376763 hab.)
Promedio de población global: 117079875.00 hab.
Promedio de superficie global: 3007791.00 km²

Cantidad de países por continente:
- América: 2
- Asia: 1
- Europa: 1
```

4. Dificultades Encontradas y Resolución

4.1 Persistencia de Datos y el Problema de las Rutas en el Entorno de Desarrollo

Dificultad: Al implementar la función `cargar_datos`, el programa arrojaba de forma constante una excepción de tipo `FileNotFoundError`, a pesar de que el archivo `países.csv` se encontraba creado. Esto se debía a que el directorio de trabajo activo de la terminal de Visual Studio Code apuntaba por defecto a la carpeta de instalación del editor y no a la ubicación física del proyecto.

Resolución: Se solucionó modificando el espacio de trabajo en el entorno de desarrollo mediante la opción "Open Folder", apuntando la terminal exactamente sobre el directorio del proyecto. Adicionalmente, se robusteció el código envolviendo la apertura del archivo en una estructura `try-except`.

4.2 Control de Integridad y Datos Duplicados en Memoria

Dificultad: La primera versión de la función `agregar_pais` permitía ingresar múltiples veces un mismo territorio de forma consecutiva, generando inconsistencias en la base de datos y alterando los cálculos de promedios globales.

Resolución: Se aplicó un algoritmo de validación previa que recorre la lista de diccionarios comparando nombres normalizados con el método `.lower()`. Si se detecta duplicado, el sistema emite un mensaje de error y frena la inserción mediante `return`.

4.3 Control de Tipos de Datos en Entradas Numéricas

Dificultad: Al solicitar la población o superficie, si el usuario ingresaba letras o caracteres especiales, la conversión directa a entero (int) interrumpía la ejecución con un ValueError.

Resolución: Se aislaron las solicitudes numéricas dentro de bloques try-except capturando específicamente ValueError. Adicionalmente, mediante condicionales se verifica que el valor sea mayor o igual a cero.

4.4 Duplicación de Código en Validaciones de Entrada

Dificultad: La estructura inicial repetía bloques idénticos de control try-except para convertir strings a enteros positivos, afectando la uniformidad y contradiciéndose con el principio DRY (Don't Repeat Yourself).

Resolución: Se encapsuló la lógica en la función auxiliar solicitar_entero_positivo(). Las funciones principales ahora invocan este módulo como servicio de validación externo, reduciendo duplicación y garantizando un diseño escalable.

4.5 Optimización de Robustez y Usabilidad mediante Pruebas de Sistema

Dificultad: Durante las pruebas de control de calidad sobre el programa, se detectó que la persistencia dependía exclusivamente de la opción de salir, arriesgando la pérdida de datos ante cierres abruptos (como un comando Ctrl+C). Asimismo, la función de estadísticas sufría un fallo crítico de desbordamiento de índice (IndexError) si el archivo CSV se encontraba vacío, el filtro por continente exigía texto exacto rompiendo la consistencia con la búsqueda general, y el menú carecía de una opción directa para visualizar el padrón completo de registros.

Resolución: Se modificó la arquitectura de persistencia para guardar los datos en el archivo CSV de forma inmediata tras cada alta o modificación exitosa. Se aplicó programación defensiva en el módulo de métricas agregando un control de lista vacía que retorna valores seguros. Adicionalmente, se reemplazó la igualdad estricta por el operador de membresía "in" en el filtro geográfico para unificar las coincidencias parciales, y se expandió la interfaz de usuario incorporando una octava opción que permite listar la totalidad de los países en formato tabular.

Asimismo, se detectó una debilidad de usabilidad en los submenús de filtrado y ordenamiento, donde un ingreso inválido del usuario interrumpía el flujo correcto. Esto se resolvió aplicando programación defensiva mediante bucles de validación indeterminados (`while True`) que restringen el avance ante datos erróneos, incorporando además una opción de escape estandarizada para retornar al menú principal de forma segura.

5. Conclusiones y Aprendizajes

La realización de este Trabajo Práctico Integrador permitió consolidar los fundamentos de la programación estructurada y el diseño algorítmico adquiridos a lo largo de la materia.

A nivel técnico, el proyecto demostró la importancia de la modularización. Dividir el sistema en funciones con responsabilidades únicas no solo facilitó la lectura y el mantenimiento del código, sino que permitió una distribución equitativa de las tareas de desarrollo dentro del equipo. Asimismo, se comprendió el valor de la persistencia de datos y el manejo de excepciones para construir software confiable y tolerante a fallas de usuario.

A nivel profesional, la utilización de Git y GitHub como herramientas de control de versiones introdujo al equipo en las metodologías de trabajo reales de la industria del software. Aprender a coordinar la subida de archivos, documentar los avances mediante commits y estructurar un repositorio público constituyó un aprendizaje tan valioso como la lógica de programación en sí misma. El trabajo colaborativo potenció la capacidad de debate técnico y la resolución conjunta de problemas complejos.

6. Bibliografía y Webgrafía

6.1 Bibliografía Académica (Libros y Apuntes de Cátedra)

- Cátedra de Programación 1. (2026). Apuntes Teóricos: Unidades 1 a 10. Tecnicatura Universitaria en Programación. Universidad Tecnológica Nacional.

- Matthes, E. (2019). Python Crash Course: A Hands-On, Project-Based Introduction to Programming (2da ed.). No Starch Press.
- Van Rossum, G., & Drake, F. L. (2009). The Python Language Reference. Python Software Foundation.

6.2 Webgrafía (Documentación Oficial y Recursos Técnicos)

- Documentación oficial del módulo CSV en Python:
- Python Software Foundation. (2026). csv — CSV File Reading and Writing. Recuperado de python.org
- Guía oficial del estilo de código PEP 8:
- Van Rossum, G., Warsaw, B., & Coghlan, N. (2013). PEP 8 – Style Guide for Python Code. Recuperado de python.org
- Documentación oficial de las estructuras de datos (Listas y Diccionarios):
- Python Software Foundation. (2026). Data Structures — Python Documentation. Recuperado de python.org
- Documentación oficial y guías de Git y GitHub:
- GitHub, Inc. (2026). GitHub Docs: Configuración y uso de repositorios remotos. Recuperado de github.com
- Chacon, S., & Straub, B. (2014). Pro Git (2da ed.). Apress. Recuperado de git-scm.com

Links del Proyecto

Repositorio GitHub (incluye código fuente, CSV y documentación PDF):

<https://github.com/lautaron-hub/tp-integrador-programacion-tupad-utn>

Video Demostrativo:

https://drive.google.com/drive/folders/1LgjrK6BeoQpA_BnzROZEor3YWy2ZnsjT